

Práctica 1: Optimización de Código

Asignatura Tópicos A

Luis Antonio Muñoz Vásquez

Universidad de Magallanes

Mayo de 2026

Facultad de Ingeniería

Departamento de Ingeniería en Computación

Punta Arenas, Chile

Índice

1. Introducción	5
2. Objetivos	5
2.1. Objetivo general	5
2.2. Objetivos específicos	5
3. Marco teórico	6
3.1. Complejidad algorítmica	6
3.1.1. Producto punto	6
3.1.2. Multiplicación matriz-vector	6
3.1.3. Multiplicación matriz-matriz	6
3.2. GNU Octave	7
3.3. CBLAS	7
3.4. Optimización de código	7
4. Metodología	8
4.1. Entorno de trabajo	8
4.2. Procedimiento de medición	8
4.3. Tamaños utilizados	8
5. Implementaciones	9
5.1. Producto punto	9
5.2. Multiplicación matriz-vector	9
5.3. Multiplicación matriz-matriz	10
5.4. Relación entre implementación y complejidad algorítmica	11
5.4.1. Producto punto — $O(n)$	11
5.4.2. Multiplicación matriz-vector — $O(n^2)$	11
5.4.3. Multiplicación matriz-matriz — $O(n^3)$	12

6. Resultados	12
6.1. Comparación Octave vs CBLAS	12
6.1.1. Producto punto	12
6.1.2. Multiplicación matriz-vector	13
6.1.3. Multiplicación matriz-matriz	14
6.2. Comparación C manual / C optimizado / CBLAS	14
6.2.1. Producto punto	14
6.2.2. Multiplicación matriz-vector	15
6.2.3. Multiplicación matriz-matriz	16
7. Análisis de resultados	18
7.1. Análisis del producto punto	18
7.2. Análisis de la multiplicación matriz-vector	18
7.3. Análisis de la multiplicación matriz-matriz	18
8. Discusión	19
9. Conclusiones	19
10. Anexos	20
10.1. Estructura de carpetas	20
10.2. Código principal de producto punto en C	21
10.3. Código optimizado de producto punto en C	23
10.4. Código principal de matriz-vector en C	25
10.5. Código optimizado de matriz-vector en C	28
10.6. Código principal de matriz-matriz en C	31
10.7. Código optimizado de matriz-matriz en C	33
10.8. Script de comparación para producto punto	36
10.9. Script de comparación para matriz-vector	38

10.10Script de comparación para matriz-matriz 39

1. Introducción

La presente práctica tiene como finalidad estudiar el rendimiento computacional de distintas operaciones de álgebra lineal, comparando implementaciones realizadas en GNU Octave, lenguaje C y funciones de la biblioteca CBLAS.

Las operaciones evaluadas fueron el producto punto, la multiplicación matriz-vector y la multiplicación matriz-matriz. Estas operaciones fueron seleccionadas debido a que representan distintos órdenes de complejidad algorítmica: lineal, cuadrático y cúbico, respectivamente.

El trabajo se desarrolló en un entorno WSL, utilizando herramientas de compilación y medición como GCC, Octave, CBLAS, Bash y Gnuplot. Se implementaron versiones manuales en C, versiones optimizadas y se compararon con rutinas especializadas de CBLAS.

El objetivo central fue observar cómo influyen la complejidad algorítmica, el tipo de lenguaje, la optimización de código y la localidad de memoria sobre los tiempos de ejecución.

2. Objetivos

2.1. Objetivo general

Comparar el rendimiento de operaciones de álgebra lineal implementadas en Octave, C manual, C optimizado y CBLAS, evaluando el efecto de la complejidad algorítmica y de las técnicas de optimización de código.

2.2. Objetivos específicos

- Implementar y medir el tiempo de ejecución del producto punto, multiplicación matriz-vector y multiplicación matriz-matriz.
- Comparar operadores nativos de Octave con implementaciones propias.
- Implementar las mismas operaciones en lenguaje C.
- Aplicar técnicas de optimización como loop unrolling y cambio de orden de ciclos.
- Comparar las implementaciones manuales y optimizadas con funciones CBLAS.
- Analizar los resultados obtenidos a partir de tablas y gráficas.

3. Marco teórico

3.1. Complejidad algorítmica

La complejidad algorítmica permite estudiar cómo crece el costo computacional de un algoritmo a medida que aumenta el tamaño del problema. En esta práctica se analizaron tres operaciones fundamentales.

3.1.1. Producto punto

El producto punto entre dos vectores de tamaño n se define como:

$$c = \sum_{i=1}^n x_i y_i \quad (1)$$

La operación requiere recorrer los vectores una vez, por lo que su complejidad es:

$$O(n) \quad (2)$$

3.1.2. Multiplicación matriz-vector

La multiplicación entre una matriz $A \in \mathbb{R}^{n \times n}$ y un vector $x \in \mathbb{R}^n$ se define como:

$$y_i = \sum_{j=1}^n A_{ij} x_j \quad (3)$$

Esta operación requiere dos ciclos anidados, por lo que su complejidad es:

$$O(n^2) \quad (4)$$

3.1.3. Multiplicación matriz-matriz

La multiplicación entre dos matrices cuadradas A y B de tamaño $n \times n$ se define como:

$$C_{ij} = \sum_{k=1}^n A_{ik} B_{kj} \quad (5)$$

Esta operación requiere tres ciclos anidados, por lo que su complejidad es:

$$O(n^3) \tag{6}$$

3.2. GNU Octave

GNU Octave permite resolver operaciones de álgebra lineal mediante operadores nativos. Estos operadores suelen estar asociados a rutinas compiladas y optimizadas, por lo que pueden presentar tiempos de ejecución considerablemente menores que una función escrita directamente en Octave mediante ciclos interpretados.

3.3. CBLAS

CBLAS corresponde a una interfaz en lenguaje C para acceder a rutinas BLAS. Estas rutinas están diseñadas para operaciones de álgebra lineal y suelen estar altamente optimizadas.

En esta práctica se utilizaron las siguientes funciones:

- `cblas_ddot`: producto punto.
- `cblas_dgemv`: multiplicación matriz-vector.
- `cblas_dgemm`: multiplicación matriz-matriz.

3.4. Optimización de código

La optimización de código busca reducir el tiempo de ejecución de un programa sin modificar el resultado final. En esta práctica se aplicaron dos estrategias principales:

- **Loop unrolling**: técnica que reduce la sobrecarga de control de ciclos, realizando varias operaciones por iteración.
- **Cambio de orden de ciclos**: técnica orientada a mejorar el acceso a memoria y la localidad de datos, especialmente relevante en operaciones con matrices.

4. Metodología

4.1. Entorno de trabajo

El desarrollo se realizó en WSL utilizando las siguientes herramientas:

- GNU Octave para implementar y medir operaciones interpretadas.
- GCC para compilar programas en C.
- CBLAS mediante la biblioteca `libblas-dev`.
- Bash para automatizar las mediciones.
- Gnuplot para generar las gráficas.
- LaTeX para elaborar el informe.

4.2. Procedimiento de medición

Para cada tamaño del problema se ejecutó cada algoritmo siete veces. Luego, se calculó el promedio de los tiempos obtenidos. Esta estrategia permite reducir el efecto de variaciones puntuales del sistema operativo o de la carga de CPU.

En Octave se utilizaron las funciones `tic` y `toc`. En C se utilizó la función `gettimeofday`. Los datos fueron almacenados en archivos `.dat` y posteriormente graficados con Gnuplot.

4.3. Tamaños utilizados

Los tamaños de entrada fueron seleccionados de forma progresiva, usando potencias de dos. Los tamaños de entrada fueron seleccionados experimentalmente hasta encontrar rangos en los cuales tanto Octave como CBLAS entregaran tiempos de ejecución medibles y mayores a 0,01 segundos. Este criterio se utilizó para evitar mediciones poco representativas asociadas a tiempos demasiado pequeños.

5. Implementaciones

5.1. Producto punto

Para el producto punto se implementaron las siguientes versiones:

- Función manual en Octave.
- Operador nativo de Octave.
- Implementación manual en C.
- Implementación optimizada en C mediante loop unrolling.
- Función `cblas_ddot`.

La versión manual en C utiliza un ciclo simple:

```
1 for(i = 0; i < n; i++)  
2     c += x[i] * y[i];
```

La versión optimizada realiza varias operaciones dentro de una misma iteración:

```
1 for(i = 0; i < limit; i += 5)  
2 {  
3     c += x[i] * y[i]  
4         + x[i + 1] * y[i + 1]  
5         + x[i + 2] * y[i + 2]  
6         + x[i + 3] * y[i + 3]  
7         + x[i + 4] * y[i + 4];  
8 }
```

5.2. Multiplicación matriz-vector

La implementación optimizada de la multiplicación matriz-vector se presentó considerando el efecto del orden de almacenamiento en memoria. En lenguaje C los arreglos bidimensionales se almacenan en *row major order*, por lo que el recorrido por filas favorece la localidad espacial y reduce accesos ineficientes a memoria.

La **versión C row major** (manual) utiliza dos ciclos anidados recorriendo por filas:

```

1  for(i = 0; i < m; i++)
2  {
3      y[i] = 0.0;
4      for(j = 0; j < n; j++)
5      {
6          y[i] += A[i * n + j] * x[j];
7      }
8  }

```

La **versión C column major** (optimizada) reordena los accesos para comparar el efecto de la localidad de datos al recorrer la matriz en distinto orden, haciendo evidente que el rendimiento se ve afectado por el patrón de acceso a memoria.

5.3. Multiplicación matriz-matriz

En la multiplicación matriz-matriz se evaluó una **versión C clásica** y una **versión C optimizada** mediante cambio de orden de ciclos. Esta reorganización permite recorrer elementos contiguos en memoria, mejorando la localidad de datos y el rendimiento general de la implementación en C.

La implementación manual se realizó mediante tres ciclos anidados en el orden clásico:

```

1  for(i = 0; i < m; i++)
2  {
3      for(j = 0; j < n; j++)
4      {
5          double sum = 0.0;
6          for(p = 0; p < k; p++)
7          {
8              sum += A[i * k + p] * B[p * n + j];
9          }
10         C[i * n + j] = sum;
11     }
12 }

```

La versión optimizada utiliza el orden de ciclos i, p, j , lo que permite recorrer de forma más contigua la matriz B y la matriz C en memoria:

```

1  for(i = 0; i < m; i++)

```

```

2 {
3     for(p = 0; p < k; p++)
4     {
5         double a_ip = A[i * k + p];
6         int rowB = p * n;
7         int rowC = i * n;
8
9         for(j = 0; j < n; j++)
10        {
11            C[rowC + j] += a_ip * B[rowB + j];
12        }
13    }
14 }

```

5.4. Relación entre implementación y complejidad algorítmica

En esta sección se identifica, directamente en el código C, el origen de la complejidad de cada operación.

5.4.1. Producto punto — $O(n)$

La complejidad $O(n)$ se observa en la presencia de un único ciclo que recorre los n elementos de los vectores:

```

1 for(i = 0; i < n; i++)
2     c += x[i] * y[i];

```

5.4.2. Multiplicación matriz-vector — $O(n^2)$

La complejidad $O(n^2)$ se aprecia en dos ciclos anidados: uno para las filas de la matriz y otro para las columnas:

```

1 for(i = 0; i < m; i++)
2     for(j = 0; j < n; j++)
3         y[i] += A[i * n + j] * x[j];

```

5.4.3. Multiplicación matriz-matriz — $O(n^3)$

La complejidad $O(n^3)$ se identifica en tres ciclos anidados, correspondientes al cálculo de cada elemento de la matriz resultante:

```
1 for(i = 0; i < m; i++)
2     for(j = 0; j < n; j++)
3         for(p = 0; p < k; p++)
4             C[i*n+j] += A[i*k+p] * B[p*n+j];
```

6. Resultados

6.1. Comparación Octave vs CBLAS

6.1.1. Producto punto

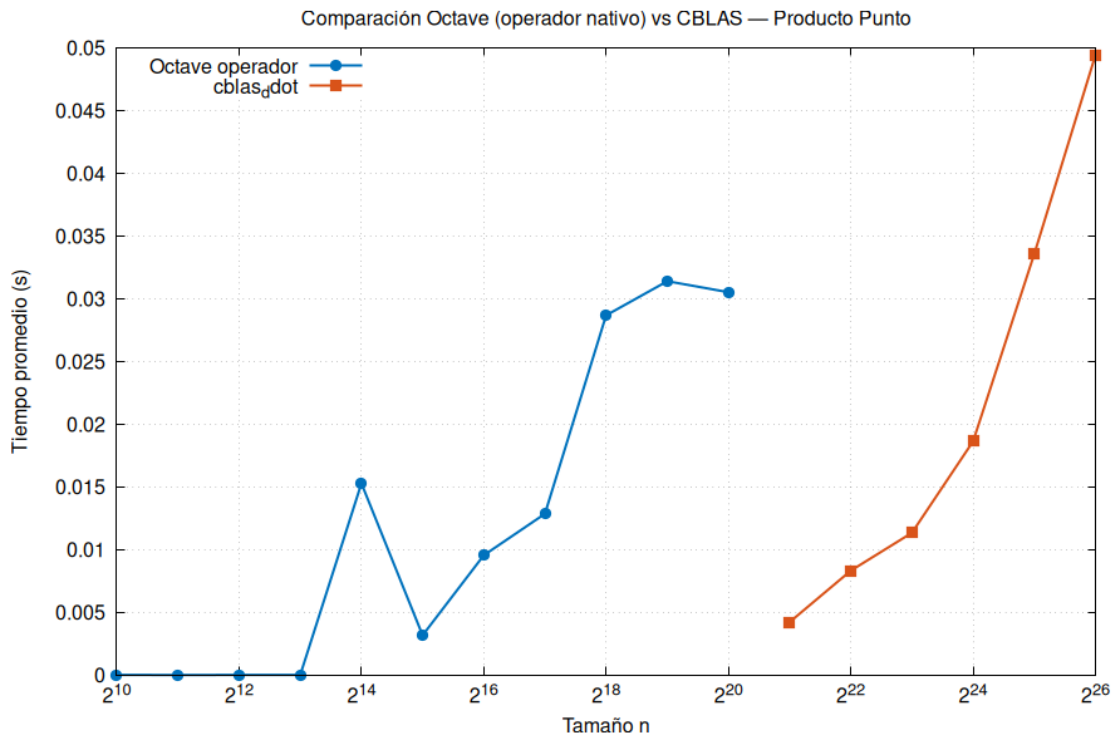


Figura 1: Comparación del operador nativo de Octave y `cblas_ddot` para el producto punto.

6.1.2. Multiplicación matriz-vector

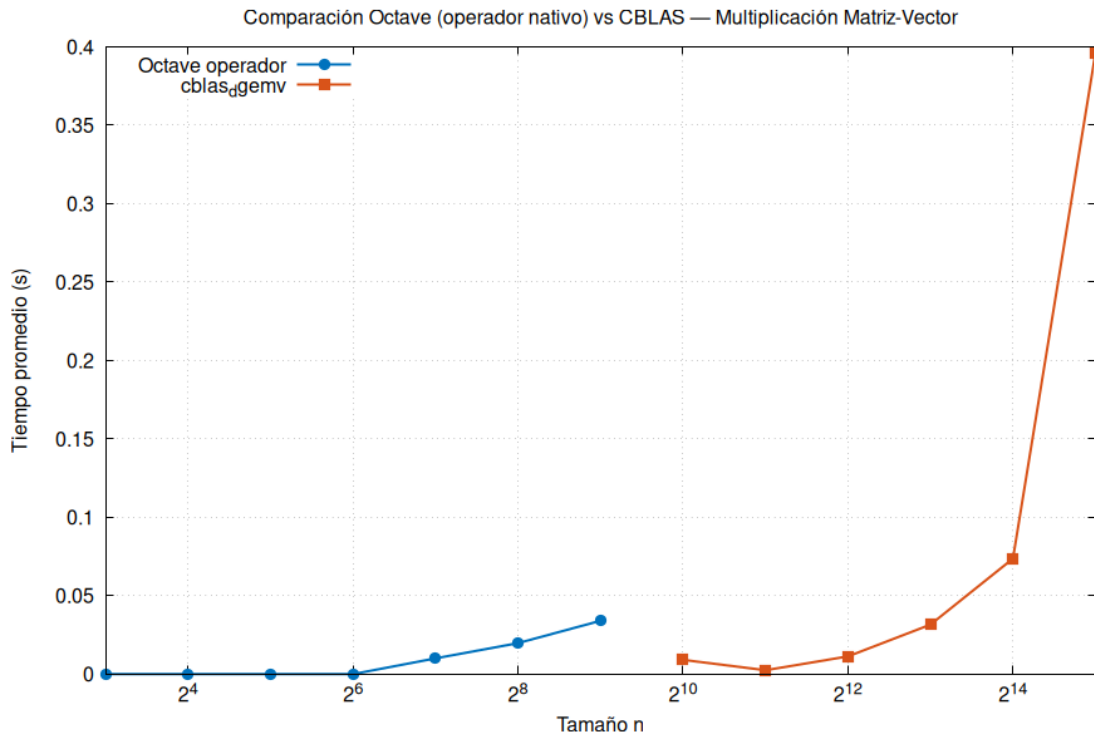


Figura 2: Comparación del operador nativo de Octave y `cblas_dgemv` para la multiplicación matriz-vector.

6.1.3. Multiplicación matriz-matriz

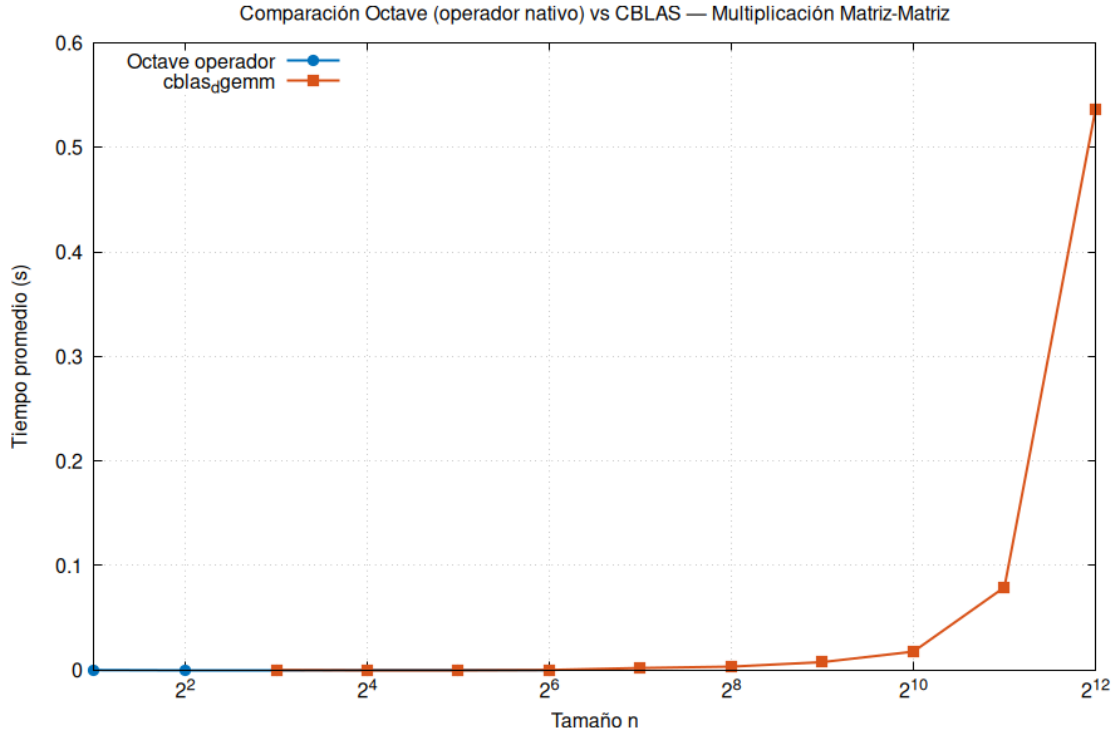


Figura 3: Comparación del operador nativo de Octave y `cblas_dgemm` para la multiplicación matriz-matriz.

6.2. Comparación C manual / C optimizado / CBLAS

6.2.1. Producto punto

Tabla 1: Tiempos de ejecución para producto punto.

n	C clásico	C loop unrolling	CBLAS
2097152	0.00197586	0.00169957	0.00213343
4194304	0.00422486	0.00346971	0.00475314
8388608	0.00798214	0.00614800	0.00580186
16777216	0.01624690	0.01267610	0.00743729
33554432	0.03522290	0.02433260	0.01515530
67108864	0.06923540	0.04843470	0.02964730

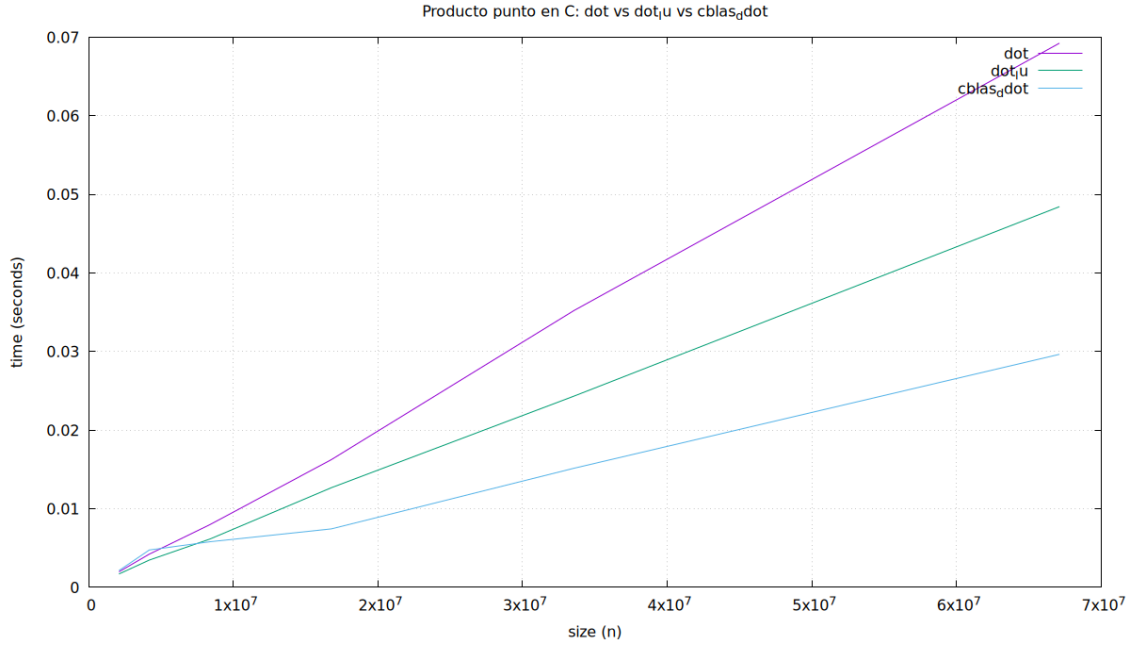


Figura 4: Comparación entre producto punto: C clásico, C loop unrolling y CBLAS.

6.2.2. Multiplicación matriz-vector

Tabla 2: Tiempos de ejecución para multiplicación matriz-vector.

n	C row major	C column major	CBLAS
1024	0.00102200	0.00067200	0.00296014
2048	0.00354857	0.00262414	0.00209571
4096	0.01483600	0.00998100	0.00576314
8192	0.05712200	0.03939500	0.01758730
16384	0.05582830	0.15424700	0.06593760
32768	1.12646000	0.99062700	0.36263900

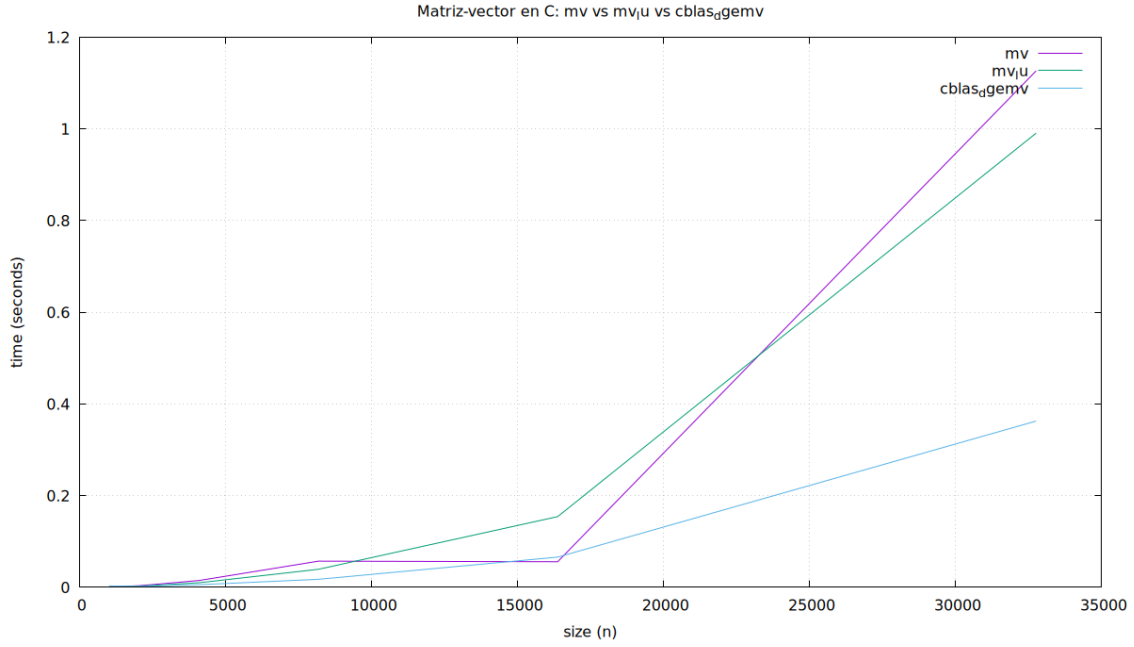


Figura 5: Comparación entre multiplicación matriz-vector: C row major, C column major y CBLAS.

6.2.3. Multiplicación matriz-matriz

Tabla 3: Tiempos de ejecución para multiplicación matriz-matriz.

n	C row major	C column major	CBLAS
n	C manual	C optimizado	CBLAS
128	0.00263500	0.000505571	0.00109614
256	0.03306990	0.002533000	0.00257829
512	0.81541800	0.018950000	0.00643086
1024	7.14204000	0.162721000	0.02086140
2048	174.15500000	2.759360000	0.06636690
4096	4219.46000000	22.464200000	0.38605000

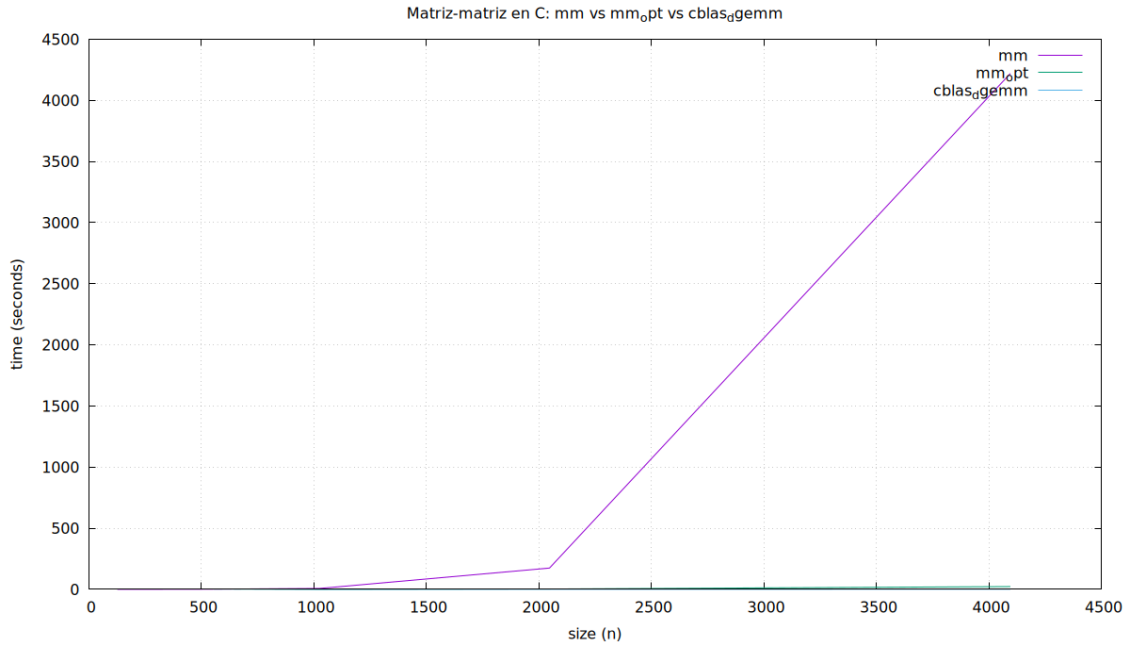


Figura 6: Comparación entre multiplicación matriz-matriz: C row major, C column major y CBLAS.

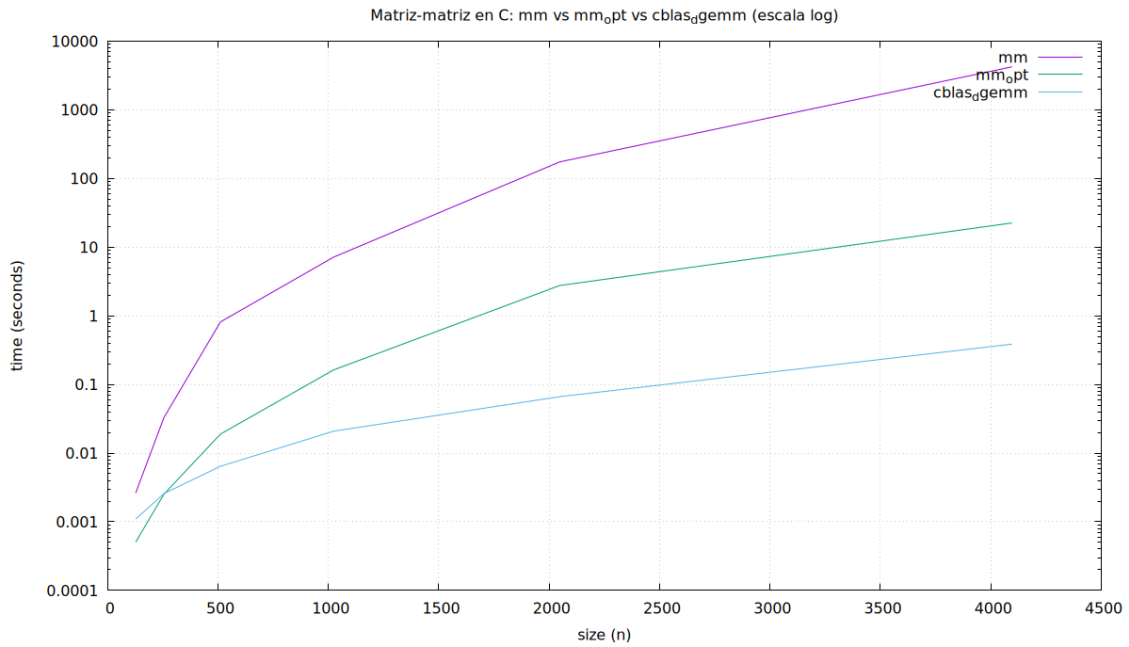


Figura 7: Comparación matriz-matriz en escala logarítmica.

7. Análisis de resultados

Los resultados muestran que las rutinas especializadas de CBLAS presentan, en general, menores tiempos de ejecución que las implementaciones manuales. Esto se explica por el uso de bibliotecas compiladas y optimizadas para operaciones de álgebra lineal. En el caso de las implementaciones en C, las mejoras obtenidas mediante loop unrolling y reorganización de ciclos evidencian que la optimización de código y la localidad de memoria influyen directamente en el rendimiento.

7.1. Análisis del producto punto

El producto punto presentó un crecimiento aproximadamente lineal, coherente con su complejidad $O(n)$. El operador nativo de Octave resultó más rápido que la función manual escrita en Octave, ya que el operador interno está implementado en código compilado. En las implementaciones en C, la versión optimizada mediante loop unrolling redujo el tiempo de ejecución al disminuir la sobrecarga de control del ciclo. A medida que el tamaño del problema aumentó, CBLAS presentó los mejores resultados, confirmando el efecto de su alto nivel de optimización.

7.2. Análisis de la multiplicación matriz-vector

La multiplicación matriz-vector presentó un crecimiento superior al producto punto, coherente con su complejidad $O(n^2)$. En esta operación, el orden de almacenamiento en memoria tiene un impacto directo: la versión C row major, al recorrer los datos de forma contigua en memoria, favorece la localidad espacial y reduce accesos ineficientes a caché. La versión CBLAS fue la más eficiente, especialmente en tamaños grandes.

Se observó una medición irregular en $n = 16384$, donde la versión optimizada presentó un tiempo mayor al esperado. Esta irregularidad puede estar asociada a ruido del sistema, carga del procesador o efectos de caché y debe interpretarse como una medición atípica.

7.3. Análisis de la multiplicación matriz-matriz

La multiplicación matriz-matriz fue la operación con mayor diferencia entre las implementaciones, resultado esperado dada su complejidad $O(n^3)$. La versión C clásica presentó tiempos muy elevados especialmente para tamaños grandes, por el acceso menos eficiente a memoria.

La versión C optimizada mediante cambio de orden de ciclos reorganizó los accesos para recorrer elementos contiguos en memoria, mejorando la localidad de datos: para $n = 4096$, la versión manual tardó 4219.46 s mientras que la optimizada tardó 22.46 s.

En comparación, `cblas_dgemm` alcanzó 0.386 s para el mismo tamaño, evidenciando el alto nivel de optimización de BLAS para operaciones matriciales. En cuanto a la comparación Octave operador vs CBLAS, CBLAS superó al operador nativo en los tamaños donde ambas mediciones resultaron representativas.

8. Discusión

Los resultados obtenidos permiten observar que la elección de la implementación influye directamente en el rendimiento. Aunque todos los algoritmos resuelven matemáticamente el mismo problema, sus tiempos de ejecución pueden variar de forma considerable.

En Octave, los operadores nativos fueron mucho más eficientes que las funciones implementadas manualmente con ciclos, debido a que las funciones escritas directamente en Octave son interpretadas, mientras que los operadores nativos se apoyan en rutinas compiladas. Sin embargo, CBLAS superó al operador nativo en las operaciones de mayor escala, lo que confirma la ventaja de las bibliotecas especializadas.

En C, las optimizaciones aplicadas permitieron mejorar el rendimiento respecto de las versiones manuales. El loop unrolling redujo la sobrecarga de control en el producto punto. El cambio de orden de ciclos en la multiplicación matriz-matriz y el uso del recorrido row major en la multiplicación matriz-vector tuvieron un impacto significativo al mejorar la localidad de memoria.

Finalmente, las funciones CBLAS demostraron el mejor rendimiento general, ya que BLAS está diseñado específicamente para operaciones de álgebra lineal y aprovecha optimizaciones internas que son difíciles de igualar con implementaciones manuales.

9. Conclusiones

Se concluye que la complejidad algorítmica determina el crecimiento general del tiempo de ejecución, pero no explica por sí sola las diferencias de rendimiento entre implementaciones. El producto punto presentó un crecimiento lineal, la multiplicación matriz-vector cuadrático y la multiplicación matriz-matriz cúbico.

Las rutinas CBLAS obtuvieron los mejores resultados en la mayoría de los casos debido a su alto nivel de optimización. Las versiones optimizadas en C mejoraron respecto de las implementaciones manuales, especialmente cuando se aplicaron técnicas como loop unrolling y reorganización del acceso a memoria.

En el producto punto, el loop unrolling redujo la sobrecarga de control del ciclo y mejoró el rendimiento de forma consistente. En la multiplicación matriz-vector, la consideración del almacenamiento row major permitió aprovechar la localidad espacial de los datos. En la multiplicación matriz-matriz, el cambio de orden de ciclos fue la optimización más determinante, reduciendo los tiempos en varios órdenes de magnitud.

En consecuencia, el rendimiento depende tanto del orden de complejidad como de la forma concreta en que el algoritmo interactúa con la arquitectura. La práctica permitió verificar experimentalmente que la optimización de código y la localidad de memoria son factores tan relevantes como la complejidad algorítmica al momento de evaluar el rendimiento computacional real.

10. Anexos

10.1. Estructura de carpetas

```
1 practical/
2 |-- octave/
3 |-- cblas/
4 |   |-- array/
5 |   |-- ddot/
6 |   |-- dgemv/
7 |   |-- dgemm/
8 |-- c/
9 |   |-- dot/
10 |   |-- mv/
11 |   |-- mm/
12 |-- resultados/
13 |   |-- datos/
14 |   |-- graficos/
15 |-- informe_latex/
```

10.2. Código principal de producto punto en C

```
1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static double my_dot(const int n, const double* x, const double* y);
16 static int main_dot(const int n, const int elements_type, const char
    * verbose);
17
18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "dot: manual C program for dot product.\n");
21     fputc('\n', stdout);
22
23     if(argc != 4)
24     {
25         fprintf(stdout, "Use: dot <n:int> <0|1|2> <on|off>.\n");
26         return EXIT_FAILURE;
27     }
28
29     main_dot(atoi(argv[1]), atoi(argv[2]), argv[3]);
30     return EXIT_SUCCESS;
31 }
32
33 static double my_dot(const int n, const double* x, const double* y)
34 {
35     double c = 0.0;
36     int i = 0;
```

```

37
38     for(i = 0; i < n; i++)
39         c += x[i] * y[i];
40
41     return c;
42 }
43
44 static int main_dot(const int n, const int elements_type, const char
    * verbose)
45 {
46     double c = 0.0;
47     double* x = NULL;
48     double* y = NULL;
49     struct timeval start, finish;
50     double runtime = 0.0;
51
52     assert(n > 0);
53     assert(elements_type >= ZEROS && elements_type <= RAND);
54
55     x = array_new(n, 1, elements_type);
56     assert(x != NULL);
57
58     y = array_new(n, 1, elements_type);
59     assert(y != NULL);
60
61     gettimeofday(&start, NULL);
62     c = my_dot(n, x, y);
63     gettimeofday(&finish, NULL);
64
65     if(strcmp(verbose, "on") == 0)
66     {
67         array_show(n, 1, x, "x");
68         array_show(n, 1, y, "y");
69     }
70
71     fprintf(stdout, "c = %lf\n", c);
72     runtime = timeval_diff(&finish, &start);
73     fprintf(stdout, "Data: %d %lf\n", n, runtime);

```

```

74
75     free(x);
76     free(y);
77
78     return EXIT_SUCCESS;
79 }

```

10.3. Código optimizado de producto punto en C

```

1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static double my_dot_lu(const int n, const double* x, const double*
    y);
16 static int main_dot_lu(const int n, const int elements_type, const
    char* verbose);
17
18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "dot_lu: manual C program for dot product with
        loop unrolling.\n");
21     fputc('\n', stdout);
22
23     if(argc != 4)
24     {
25         fprintf(stdout, "Use: dot_lu <n:int> <0|1|2> <on|off>.\n");
26         return EXIT_FAILURE;

```

```

27     }
28
29     main_dot_lu(atoi(argv[1]), atoi(argv[2]), argv[3]);
30     return EXIT_SUCCESS;
31 }
32
33 static double my_dot_lu(const int n, const double* x, const double*
    y)
34 {
35     double c = 0.0;
36     int i = 0;
37     int limit = n - (n % 5);
38
39     for(i = 0; i < limit; i += 5)
40     {
41         c += x[i] * y[i]
42             + x[i + 1] * y[i + 1]
43             + x[i + 2] * y[i + 2]
44             + x[i + 3] * y[i + 3]
45             + x[i + 4] * y[i + 4];
46     }
47
48     for(i = limit; i < n; i++)
49     {
50         c += x[i] * y[i];
51     }
52
53     return c;
54 }
55
56 static int main_dot_lu(const int n, const int elements_type, const
    char* verbose)
57 {
58     double c = 0.0;
59     double* x = NULL;
60     double* y = NULL;
61     struct timeval start, finish;
62     double runtime = 0.0;

```



```

63
64     assert(n > 0);
65     assert(elements_type >= ZEROS && elements_type <= RAND);
66
67     x = array_new(n, 1, elements_type);
68     assert(x != NULL);
69
70     y = array_new(n, 1, elements_type);
71     assert(y != NULL);
72
73     gettimeofday(&start, NULL);
74     c = my_dot_lu(n, x, y);
75     gettimeofday(&finish, NULL);
76
77     if(strcmp(verbose, "on") == 0)
78     {
79         array_show(n, 1, x, "x");
80         array_show(n, 1, y, "y");
81     }
82
83     fprintf(stdout, "c = %lf\n", c);
84     runtime = timeval_diff(&finish, &start);
85     fprintf(stdout, "Data: %d %lf\n", n, runtime);
86
87     free(x);
88     free(y);
89
90     return EXIT_SUCCESS;
91 }

```

10.4. Código principal de matriz-vector en C

```

1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"

```

```

7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static void my_mv(const int m, const int n, const double* A, const
    double* x, double* y);
16 static int main_mv(const int m, const int n, const int elements_type
    , const char* verbose);
17
18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "mv: manual C program for matrix-vector
        multiplication.\n");
21     fputc('\n', stdout);
22
23     if(argc != 5)
24     {
25         fprintf(stdout, "Use: mv <m:int> <n:int> <0|1|2> <on|off>.\n
            ");
26         return EXIT_FAILURE;
27     }
28
29     main_mv(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]), argv[4]);
30     return EXIT_SUCCESS;
31 }
32
33 static void my_mv(const int m, const int n, const double* A, const
    double* x, double* y)
34 {
35     int i, j;
36
37     for(i = 0; i < m; i++)
38     {
39         y[i] = 0.0;

```

```

40         for(j = 0; j < n; j++)
41         {
42             y[i] += A[i * n + j] * x[j];
43         }
44     }
45 }
46
47 static int main_mv(const int m, const int n, const int elements_type
48 , const char* verbose)
49 {
50     double* A = NULL;
51     double* x = NULL;
52     double* y = NULL;
53     struct timeval start, finish;
54     double runtime = 0.0;
55
56     assert(m > 0);
57     assert(n > 0);
58     assert(elements_type >= ZEROS && elements_type <= RAND);
59
60     A = array_new(m, n, elements_type);
61     assert(A != NULL);
62
63     x = array_new(n, 1, elements_type);
64     assert(x != NULL);
65
66     y = array_new(m, 1, ZEROS);
67     assert(y != NULL);
68
69     gettimeofday(&start, NULL);
70     my_mv(m, n, A, x, y);
71     gettimeofday(&finish, NULL);
72
73     if(strcmp(verbose, "on") == 0)
74     {
75         array_show(m, n, A, "A");
76         array_show(n, 1, x, "x");
77     }

```

```

77
78     array_show(m, 1, y, "y");
79     runtime = timeval_diff(&finish, &start);
80     fprintf(stdout, "Data: %d %lf\n", m, runtime);
81
82     free(A);
83     free(x);
84     free(y);
85
86     return EXIT_SUCCESS;
87 }

```

10.5. Código optimizado de matriz-vector en C

```

1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static void my_mv_lu(const int m, const int n, const double* A,
16                     const double* x, double* y);
17
18 static int main_mv_lu(const int m, const int n, const int
19                      elements_type, const char* verbose);
20
21 int main(int argc, char** argv)
22 {
23     fprintf(stdout, "mv_lu: manual C program for matrix-vector
24             multiplication with loop unrolling.\n");
25     fputc('\n', stdout);

```

```

22
23     if(argc != 5)
24     {
25         fprintf(stdout, "Use: mv_lu <m:int> <n:int> <0|1|2> <on|off\n");
26         return EXIT_FAILURE;
27     }
28
29     main_mv_lu(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]), argv[4])
30     ;
31     return EXIT_SUCCESS;
32 }
33 static void my_mv_lu(const int m, const int n, const double* A,
34     const double* x, double* y)
35 {
36     int i, j;
37     int limit = n - (n % 4);
38
39     for(i = 0; i < m; i++)
40     {
41         double sum = 0.0;
42         int base = i * n;
43
44         for(j = 0; j < limit; j += 4)
45         {
46             sum += A[base + j] * x[j]
47                 + A[base + j + 1] * x[j + 1]
48                 + A[base + j + 2] * x[j + 2]
49                 + A[base + j + 3] * x[j + 3];
50         }
51
52         for(j = limit; j < n; j++)
53         {
54             sum += A[base + j] * x[j];
55         }
56
57         y[i] = sum;

```

```

57     }
58 }
59
60 static int main_mv_lu(const int m, const int n, const int
    elements_type, const char* verbose)
61 {
62     double* A = NULL;
63     double* x = NULL;
64     double* y = NULL;
65     struct timeval start, finish;
66     double runtime = 0.0;
67
68     assert(m > 0);
69     assert(n > 0);
70     assert(elements_type >= ZEROS && elements_type <= RAND);
71
72     A = array_new(m, n, elements_type);
73     assert(A != NULL);
74
75     x = array_new(n, 1, elements_type);
76     assert(x != NULL);
77
78     y = array_new(m, 1, ZEROS);
79     assert(y != NULL);
80
81     gettimeofday(&start, NULL);
82     my_mv_lu(m, n, A, x, y);
83     gettimeofday(&finish, NULL);
84
85     if(strcmp(verbose, "on") == 0)
86     {
87         array_show(m, n, A, "A");
88         array_show(n, 1, x, "x");
89     }
90
91     array_show(m, 1, y, "y");
92     runtime = timeval_diff(&finish, &start);
93     fprintf(stdout, "Data: %d %lf\n", m, runtime);

```

```

94
95     free(A);
96     free(x);
97     free(y);
98
99     return EXIT_SUCCESS;
100 }

```

10.6. Código principal de matriz-matriz en C

```

1  #ifndef _GNU_SOURCE
2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static void my_mm(const int m, const int n, const int k, const
16                  double* A, const double* B, double* C);
17
18 static int main_mm(const int m, const int n, const int k, const int
19                  elements_type, const char* verbose);
20
21 int main(int argc, char** argv)
22 {
23     fprintf(stdout, "mm: manual C program for matrix-matrix
24             multiplication.\n");
25     fputc('\n', stdout);
26
27     if(argc != 6)
28     {

```

```

25     fprintf(stdout, "Use: mm <m:int> <n:int> <k:int> <0|1|2> <on
    |off>.\n");
26     return EXIT_FAILURE;
27 }
28
29     main_mm(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]), atoi(argv
    [4]), argv[5]);
30     return EXIT_SUCCESS;
31 }
32
33 static void my_mm(const int m, const int n, const int k, const
    double* A, const double* B, double* C)
34 {
35     int i, j, p;
36
37     for(i = 0; i < m; i++)
38     {
39         for(j = 0; j < n; j++)
40         {
41             double sum = 0.0;
42             for(p = 0; p < k; p++)
43             {
44                 sum += A[i * k + p] * B[p * n + j];
45             }
46             C[i * n + j] = sum;
47         }
48     }
49 }
50
51 static int main_mm(const int m, const int n, const int k, const int
    elements_type, const char* verbose)
52 {
53     double* A = NULL;
54     double* B = NULL;
55     double* C = NULL;
56     struct timeval start, finish;
57     double runtime = 0.0;
58

```



```

59     assert(m > 0);
60     assert(n > 0);
61     assert(k > 0);
62     assert(elements_type >= ZEROS && elements_type <= RAND);
63
64     A = array_new(m, k, elements_type);
65     assert(A != NULL);
66
67     B = array_new(k, n, elements_type);
68     assert(B != NULL);
69
70     C = array_new(m, n, ZEROS);
71     assert(C != NULL);
72
73     gettimeofday(&start, NULL);
74     my_mm(m, n, k, A, B, C);
75     gettimeofday(&finish, NULL);
76
77     if(strcmp(verbose, "on") == 0)
78     {
79         array_show(m, k, A, "A");
80         array_show(k, n, B, "B");
81     }
82
83     array_show(m, n, C, "C");
84     runtime = timeval_diff(&finish, &start);
85     fprintf(stdout, "Data: %d %lf\n", m, runtime);
86
87     free(A);
88     free(B);
89     free(C);
90
91     return EXIT_SUCCESS;
92 }

```

10.7. Código optimizado de matriz-matriz en C

```

1 #ifndef _GNU_SOURCE

```

```

2  #define _GNU_SOURCE
3  #endif
4
5  #ifndef ARRAY_H
6  #include "../cblas/array/array.h"
7  #endif
8
9  #include <stdio.h>
10 #include <stdlib.h>
11 #include <assert.h>
12 #include <string.h>
13 #include <sys/time.h>
14
15 static void my_mm_opt(const int m, const int n, const int k, const
    double* A, const double* B, double* C);
16 static int main_mm_opt(const int m, const int n, const int k, const
    int elements_type, const char* verbose);
17
18 int main(int argc, char** argv)
19 {
20     fprintf(stdout, "mm_opt: optimized C program for matrix-matrix
        multiplication.\n");
21     fputc('\n', stdout);
22
23     if(argc != 6)
24     {
25         fprintf(stdout, "Use: mm_opt <m:int> <n:int> <k:int> <0|1|2>
            <on|off>.\n");
26         return EXIT_FAILURE;
27     }
28
29     main_mm_opt(atoi(argv[1]), atoi(argv[2]), atoi(argv[3]), atoi(
        argv[4]), argv[5]);
30     return EXIT_SUCCESS;
31 }
32
33 static void my_mm_opt(const int m, const int n, const int k, const
    double* A, const double* B, double* C)

```

```

34 {
35     int i, j, p;
36
37     for(i = 0; i < m; i++)
38     {
39         for(j = 0; j < n; j++)
40             C[i * n + j] = 0.0;
41     }
42
43     for(i = 0; i < m; i++)
44     {
45         for(p = 0; p < k; p++)
46         {
47             double a_ip = A[i * k + p];
48             int rowB = p * n;
49             int rowC = i * n;
50
51             for(j = 0; j < n; j++)
52             {
53                 C[rowC + j] += a_ip * B[rowB + j];
54             }
55         }
56     }
57 }
58
59 static int main_mm_opt(const int m, const int n, const int k, const
60 int elements_type, const char* verbose)
61 {
62     double* A = NULL;
63     double* B = NULL;
64     double* C = NULL;
65     struct timeval start, finish;
66     double runtime = 0.0;
67
68     assert(m > 0);
69     assert(n > 0);
70     assert(k > 0);
71     assert(elements_type >= ZEROS && elements_type <= RAND);

```

```

71
72     A = array_new(m, k, elements_type);
73     assert(A != NULL);
74
75     B = array_new(k, n, elements_type);
76     assert(B != NULL);
77
78     C = array_new(m, n, ZEROS);
79     assert(C != NULL);
80
81     gettimeofday(&start, NULL);
82     my_mm_opt(m, n, k, A, B, C);
83     gettimeofday(&finish, NULL);
84
85     if(strcmp(verbose, "on") == 0)
86     {
87         array_show(m, k, A, "A");
88         array_show(k, n, B, "B");
89     }
90
91     array_show(m, n, C, "C");
92     runtime = timeval_diff(&finish, &start);
93     fprintf(stdout, "Data: %d %lf\n", m, runtime);
94
95     free(A);
96     free(B);
97     free(C);
98
99     return EXIT_SUCCESS;
100 }

```

10.8. Script de comparación para producto punto

```

1  #!/bin/bash
2
3  i=0
4  vals[0]=2097152
5

```

```

6 while [ $i -lt 5 ]
7 do
8     valtmp=${vals[$i]}
9     newval=$(echo "$valtmp * 2" | bc)
10    i=$((i + 1))
11    vals[$i]=$newval
12 done
13
14 unset LC_ALL
15 export LC_NUMERIC=C
16
17 echo "" > dot_compare.dat
18
19 for val in "${vals[@]}"
20 do
21     echo "" > data_dot.dat
22     echo "" > data_dot_lu.dat
23     echo "" > data_cblas.dat
24
25     for i in {1..7}
26     do
27         ./dot "${val}" 1 off | awk '/^Data/ { print $2 " " $3 }' >>
            data_dot.dat
28         ./dot_lu "${val}" 1 off | awk '/^Data/ { print $2 " " $3 }'
            >> data_dot_lu.dat
29         ~/practical/cblas/ddot/ddot "${val}" 1 off | awk '/^Data/ {
            print $2 " " $3 }' >> data_cblas.dat
30         echo "${val} ${i} test finished"
31     done
32
33     t_dot=$(awk '{sum += $2} END {print sum/7}' data_dot.dat)
34     t_dot_lu=$(awk '{sum += $2} END {print sum/7}' data_dot_lu.dat)
35     t_cblas=$(awk '{sum += $2} END {print sum/7}' data_cblas.dat)
36
37     echo "${val} ${t_dot} ${t_dot_lu} ${t_cblas}" >> dot_compare.dat
38 done
39
40 awk 'NF > 0' dot_compare.dat

```

10.9. Script de comparación para matriz-vector

```
1  #!/bin/bash
2
3  i=0
4  vals[0]=1024
5
6  while [ $i -lt 5 ]
7  do
8      valtmp=${vals[$i]}
9      newval=$(echo "$valtmp * 2" | bc)
10     i=$((i + 1))
11     vals[$i]=$newval
12 done
13
14 unset LC_ALL
15 export LC_NUMERIC=C
16
17 echo "" > mv_compare.dat
18
19 for val in "${vals[@]}"
20 do
21     echo "" > data_mv.dat
22     echo "" > data_mv_lu.dat
23     echo "" > data_cblas.dat
24
25     for i in {1..7}
26     do
27         ./mv "${val}" "${val}" 1 off | awk '/^Data/ { print $2 " "
28             $3 }' >> data_mv.dat
29         ./mv_lu "${val}" "${val}" 1 off | awk '/^Data/ { print $2 "
30             " $3 }' >> data_mv_lu.dat
31         ~/practical/cblas/dgemv/dgemv "${val}" "${val}" 1 off | awk
32             '/^Data/ { print $2 " " $3 }' >> data_cblas.dat
33         echo "${val} ${i} test finished"
34     done
35
36     t_mv=$(awk '{sum += $2} END {print sum/7}' data_mv.dat)
37     t_mv_lu=$(awk '{sum += $2} END {print sum/7}' data_mv_lu.dat)
```

```

35     t_cblas=$(awk '{sum += $2} END {print sum/7}' data_cblas.dat)
36
37     echo "${val} ${t_mv} ${t_mv_lu} ${t_cblas}" >> mv_compare.dat
38 done
39
40 awk 'NF > 0' mv_compare.dat

```

10.10. Script de comparación para matriz-matriz

```

1  #!/bin/bash
2
3  i=0
4  vals[0]=128
5
6  while [ $i -lt 5 ]
7  do
8      valtmp=${vals[$i]}
9      newval=$(echo "$valtmp * 2" | bc)
10     i=$((i + 1))
11     vals[$i]=$newval
12 done
13
14 unset LC_ALL
15 export LC_NUMERIC=C
16
17 echo "" > mm_compare.dat
18
19 for val in "${vals[@]}"
20 do
21     echo "" > data_mm.dat
22     echo "" > data_mm_opt.dat
23     echo "" > data_cblas.dat
24
25     for i in {1..7}
26     do
27         ./mm "${val}" "${val}" "${val}" 1 off | awk '/^Data/ { print
            $2 " " " $3 }' >> data_mm.dat

```

```

28     ./mm_opt "${val}" "${val}" "${val}" 1 off | awk '/^Data/ {
        print $2 " " $3 }' >> data_mm_opt.dat
29     ~/practical1/cblas/dgemm/dgemm "${val}" "${val}" "${val}" 1
        off | awk '/^Data/ { print $2 " " $3 }' >> data_cblas.dat
30     echo "${val} ${i} test finished"
31 done
32
33     t_mm=$(awk '{sum += $2} END {print sum/7}' data_mm.dat)
34     t_mm_opt=$(awk '{sum += $2} END {print sum/7}' data_mm_opt.dat)
35     t_cblas=$(awk '{sum += $2} END {print sum/7}' data_cblas.dat)
36
37     echo "${val} ${t_mm} ${t_mm_opt} ${t_cblas}" >> mm_compare.dat
38 done
39
40 awk 'NF > 0' mm_compare.dat

```